

BÁO CÁO PHƯƠNG PHÁP THIẾT KẾ THUẬT TOÁN

Bài toán: Aura Tracker (Hệ thống giám sát Aura)

Nhóm 17

- 1. Bùi Huỳnh Tây - MSSV: 24521589**
- 2. Phạm Ngọc Thọ - MSSV: 24520032**

Lớp: Phân tích và thiết kế thuật toán - CS112.Q11.KHTN
Trường Đại học Công nghệ Thông tin

Ngày 20 tháng 11 năm 2025

Tóm tắt bài toán

Bài toán yêu cầu tính tổng lượng Aura trong n vùng hình chữ nhật và lượng Aura của các Hunter không thuộc vùng nào. Các vùng có thể lồng nhau, và một Hunter chỉ thuộc về vùng nhỏ nhất (sâu nhất) chứa nó.

1 Subtask 1: $n, m \leq 10^3$

1.1 Phương pháp thiết kế thuật toán: Completed search - Brute force

Với kích thước dữ liệu đầu vào nhỏ, ta sử dụng phương pháp **Vét cạn (Brute Force)**. Thuật toán duyệt qua toàn bộ không gian tìm kiếm để tìm kết quả chính xác cho từng Hunter.

1.2 Tại sao phương pháp đó phù hợp?

- **Tính đơn giản:** Không cần cài đặt cấu trúc dữ liệu phức tạp.
- **Phù hợp với ràng buộc:** Với $N, M \leq 1000$, tổng số phép toán là $N \times M = 10^6$. Máy tính có thể xử lý dễ dàng trong thời gian cho phép (thường là 10^8 phép tính/giây).

1.3 Phân tích độ phức tạp

- **Độ phức tạp thời gian:** $O(M \times N)$.
 - Với mỗi Hunter trong M hunter, ta phải duyệt qua toàn bộ danh sách N vùng Aura.
 - Tại mỗi bước duyệt, ta thực hiện 4 phép so sánh tọa độ ($X_1 \leq C_j \leq X_2$ và $Y_1 \leq R_j \leq Y_2$) và cập nhật diện tích nhỏ nhất. Đây là thao tác $O(1)$.
 - Tổng thời gian: $M \times N \times O(1) \approx 10^6$ phép tính.
- **Độ phức tạp không gian:** $O(N + M)$.
 - Cần mảng kích thước N để lưu thông tin các vùng (X, Y, A, B) .
 - Cần mảng kích thước M để lưu thông tin Hunter (C, R, t) .

1.4 Code subtask 1

```

1 def solve():
2     try:
3         line1 = sys.stdin.read().split()
4     except Exception: return
5     it = iter(line1)
6     W = int(next(it)); H = int(next(it))
7     n = int(next(it)); m = int(next(it))
8
9     zones = []
10    for _ in range(n):
11        zones.append([int(next(it)) for _ in range(4)])
12
13    ans = [0] * (n + 1)
14    for _ in range(m):
15        Cx = int(next(it)); Cy = int(next(it)); t = int(next(it))

```

```

16     best_id = -1
17     min_area = float('inf')
18
19     for i in range(n):
20         x1, y1, x2, y2 = zones[i]
21         if x1 < Cx < x2 and y1 < Cy < y2:
22             area = (x2 - x1) * (y2 - y1)
23             if area < min_area:
24                 min_area = area
25                 best_id = i
26
27     if best_id != -1:
28         ans[best_id] += t
29     else:
30         ans[n] += t
31
32     print(*(ans[:n]), ans[n])

```

Listing 1: Code mẫu Subtask 1

2 Subtask 2: $W, H \leq 10^3$

2.1 Phương pháp thiết kế thuật toán: Transform and Conquer (Representation Change)

Ta biến đổi cách biểu diễn bài toán từ danh sách tọa độ sang biểu diễn **Ma trận (2D Grid)**. Ta dùng một mảng 2 chiều để mô phỏng bản đồ.

2.2 Tại sao phương pháp đó phù hợp?

- **Giới hạn không gian nhỏ:** Do $W, H \leq 1000$, mảng 2 chiều chỉ tốn khoảng 10^6 ô nhớ. Với kiểu dữ liệu `int` (4 bytes), bộ nhớ tiêu tốn khoảng $\approx 4\text{MB}$, hoàn toàn khả thi.
- **Truy vấn tức thời:** Việc chuyển đổi sang ma trận giúp bài toán xác định vị trí Hunter trở thành truy cập mảng trực tiếp $O(1)$, thay vì phải tìm kiếm.

2.3 Phân tích độ phức tạp

- **Độ phức tạp thời gian:** $O(N \cdot W \cdot H + M)$.
 - **Bước khởi tạo (Preprocessing):** Sắp xếp các vùng theo diện tích giảm dần ($O(N \log N)$). Sau đó, với mỗi vùng i , ta duyệt qua tất cả các ô (x, y) thuộc vùng đó để gán ID lên bản đồ. Trong trường hợp xấu nhất (mọi vùng đều to bằng bản đồ), chi phí là $O(N \cdot W \cdot H)$.
 - **Bước truy vấn:** Duyệt qua M Hunter, mỗi Hunter truy cập bản đồ mất $O(1)$. Tổng là $O(M)$.
- **Độ phức tạp không gian:** $O(W \times H)$.
 - Cần một mảng 2 chiều kích thước $W \times H$ để lưu trữ ID của vùng nhỏ nhất tại mỗi tọa độ. Các mảng phụ khác tốn $O(N + M)$ không đáng kể so với lưới.

2.4 Code subtask 2

```

1 def solve():
2     import sys
3     input = sys.stdin.read
4     data = input().split()
5     iterator = iter(data)
6
7     W = int(next(iterator)); H = int(next(iterator))
8     n = int(next(iterator)); m = int(next(iterator))
9
10    zones = []
11    for i in range(n):
12        x1 = int(next(iterator)); y1 = int(next(iterator))
13        x2 = int(next(iterator)); y2 = int(next(iterator))
14        area = (x2 - x1) * (y2 - y1)
15        zones.append((area, x1, y1, x2, y2, i))
16
17    zones.sort(key=lambda x: x[0], reverse=True)
18
19    grid = [[-1] * (H + 1) for _ in range(W + 1)]
20
21    for _, x1, y1, x2, y2, idx in zones:
22        for r in range(x1 + 1, x2):
23            for c in range(y1 + 1, y2):
24                grid[r][c] = idx
25
26    ans = [0] * (n + 1)
27    for _ in range(m):
28        Cx = int(next(iterator)); Cy = int(next(iterator))
29        t = int(next(iterator))
30
31        if 0 <= Cx <= W and 0 <= Cy <= H:
32            zid = grid[Cx][Cy]
33            if zid != -1:
34                ans[zid] += t
35            else:
36                ans[n] += t
37        else:
38            ans[n] += t
39
40    print(*(ans[:n]), ans[n])

```

Listing 2: Code mẫu Subtask 2

3 Subtask 3: Không ràng buộc ($W, H \leq 10^9; n, m \leq 10^5$)

3.1 Phương pháp thiết kế thuật toán: Divide, Decrease, Transform... and Conquer

Đây là phương pháp chủ đạo để giải quyết bài toán với dữ liệu lớn, cụ thể là sự kết hợp của Transform and Conquer và Divide and Conquer:

1. Transform and Conquer (Biến đổi và Trị):

- **Instance Simplification (Rời rạc hóa):** Tọa độ W, H lên tới 10^9 là quá lớn để xử lý trực tiếp. Ta biến đổi tập tọa độ về một tập nhỏ hơn ($O(N + M)$) bằng phương pháp Rời rạc hóa (Coordinate Compression), giữ nguyên thứ tự tương đối.

- **Problem Transformation (Sweep Line):** Ta *biến đổi* bài toán hình học 2D tính thành bài toán quản lý các đoạn 1D động theo thời gian (trục X). Kỹ thuật Quét đường (Sweep Line) giúp giảm đi một chiều dữ liệu cần xử lý.

2. Divide and Conquer (Chia để trị):

- **Segment Tree:** Để quản lý các đoạn trên trục Y (sau khi biến đổi Sweep Line), ta sử dụng Segment Tree. Đây là cấu trúc dữ liệu điển hình của tư tưởng Chia để trị: Chia đoạn $[L, R]$ thành hai nửa, giải quyết trên từng nửa và tổng hợp kết quả.

3.2 Tại sao phương pháp đó phù hợp?

- **Giải quyết vấn đề tọa độ lớn:** Rời rạc hóa giúp đưa bài toán về kích thước phụ thuộc vào số lượng phần tử (N, M) thay vì kích thước không gian (W, H) .
- **Tối ưu hiệu năng:** Với $N, M = 10^5$, thuật toán cần độ phức tạp gần $O(N \log N)$. Sự kết hợp giữa Sweep Line (loại bỏ chiều X) và Segment Tree (xử lý chiều Y trong $O(\log N)$) đáp ứng hoàn hảo yêu cầu này.
- **Xử lý lồng nhau:** Segment Tree kết hợp với Stack tại mỗi node cho phép quản lý việc các vùng chồng lên nhau, dễ dàng lấy ra vùng "trên cùng" (nhỏ nhất) tại mọi thời điểm.

3.3 Phân tích độ phức tạp

Gọi K là số lượng tọa độ Y phân biệt sau khi rời rạc hóa ($K \leq 2N + M$).

- **Độ phức tạp thời gian:** $O((N + M) \log(N + M))$.
 - **Rời rạc hóa:** Sắp xếp các tọa độ Y tốn $O(K \log K) \approx O((N + M) \log(N + M))$.
 - **Sắp xếp sự kiện:** Ta có $2N$ sự kiện vùng (Mở/Đóng) và M sự kiện Hunter. Sắp xếp các sự kiện này tốn $O((2N + M) \log(2N + M))$.
 - **Xử lý sự kiện (Sweep Line):** Duyệt qua từng sự kiện, mỗi sự kiện thực hiện thao tác Update hoặc Query trên Segment Tree. Chi phí mỗi thao tác là chiều cao của cây: $O(\log K)$. Tổng cộng: $O((2N + M) \log K)$.
- **Độ phức tạp không gian:** $O(N + M)$.
 - **Segment Tree:** Cây cần quản lý K đoạn rời rạc, số node tối đa là $4 \times K \approx O(N + M)$.
 - **Danh sách sự kiện:** Lưu trữ $2N + M$ sự kiện, tốn $O(N + M)$.
 - **Mảng nén tọa độ:** Lưu K giá trị Y, tốn $O(N + M)$.